《嵌入式系统原理及应用》第二次作业

班级：智装备 241　　学号：2024306100324　　姓名：彭俊杰

作业日期：2026 年 9 月 29 日

第 1 题　英文缩写的含义

> **【题干】**写出下列英文缩写的含义：ARM、AHB、AHB-Lite、APB、PPB、DAP、MPU、FPU、NVIC、ISR、Bitband、SCS、SysTick、MSP、PC、SP、GPIO、TIM、UART、CANBUS。

缩写	全称	含义
ARM	Advanced RISC Machine	先进精简指令集机器。ARM 公司（及由其授权的 32 位 RISC 处理器内核体系），以 IP 授权模式向各芯片厂商提供内核设计
AHB	Advanced High-performance Bus	高级高性能总线。AMBA 总线体系中的高速主干总线，挂接内核、DMA、存储器等高性能主/从设备
AHB-Lite	AHB-Lite	AHB 的精简版：只保留一个 AHB 主设备（去掉多主仲裁/握手总线管理），Cortex-M3/M4 内核的 AHB 接口即为 AHB-Lite
APB	Advanced Peripheral Bus	高级外设总线。AMBA 中的低速外设总线，经桥接器挂在 AHB 之下，挂接 USART、定时器等一般外设
PPB	Private Peripheral Bus	私有外设总线。Cortex-M 内核“私有”的调试/系统控制总线（AHB-Lite 型），专接 SysTick、NVIC、SCS、调试部件等内核级外设，地址区 0xE0000000～0xE00FFFFF
DAP	Debug Access Port	调试访问端口。调试器访问芯片内部的入口，物理接口有 JTAG（Joint Test Action Group，IEEE 1149.1 边界扫描接口）和 SWD（Serial Wire Debug，串行线调试，仅需 SWDIO/SWCLK 两根线）两种
MPU	Memory Protection Unit	存储器（保护）单元。对存储区划分读/写/执行权限，越权访问触发异常，实现任务间保护（可选部件）
FPU	Floating Point Unit	浮点运算单元。硬件执行单/双精度浮点指令（Cortex-M4F 内置单精度 FPU，即 FPv4-SP）
NVIC	Nested Vectored Interrupt Controller	嵌套向量中断控制器。Cortex-M 内核中断控制器：管理中断使能/屏蔽、抢占与子优先级、中断嵌套、向量查表
ISR	Interrupt Service Routine	中断服务（子）程序。响应某一中断源而执行的程序段，执行完返回被打断处
Bitband	Bit-banding（位带）	位带技术：把位带区（SRAM 区 0x20000000 起 1MB、外设区 0x40000000 起 1MB）的每一位映射到别名区（0x22000000、0x42000000）的一个 32 位字，对别名区的一次读/写即原子地读/改/写对应位

缩写	全称	含义
SCS	System Control Space	系统控制空间。PPB 中 0xE000E000～0xE000EFFF 区域，集中了 SysTick、NVIC、SCB（系统控制块）、MPU 等内核寄存器
SysTick	System Tick Timer	系统节拍定时器。内核自带的 24 位向下递减计数定时器（地址 0xE000E010），常用于产生 OS 时基/毫秒延时
MSP	Main Stack Pointer	主堆栈指针。Cortex-M 两个堆栈指针之一：复位后及异常处理时使用（另一为进程堆栈指针 PSP，RTOS 任务用）
PC	Program Counter	程序计数器。存放下一条取指指令地址，R13～R15 中 R15 即 PC
SP	Stack Pointer	堆栈指针。指向当前栈顶（R13），Cortex-M 中 SP = MSP 或 PSP 二选一
GPIO	General Purpose Input/Output	通用输入/输出端口。可由软件配置方向、上下拉、速度、复用功能等的引脚
TIM	Timer	定时器（STM32 命名，如 TIM2）。计数/定时、PWM、输入捕获、编码器接口等
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发器。异步串行通信接口（STM32 的 USART 去掉同步模式即 UART 功能）
CANBUS	Controller Area Network Bus	控制器局域网总线。多主、差分两线（CANH/CANL）、基于报文 ID 仲裁的串行总线，广泛用于汽车与工业控制

第 2 题　Cortex-M4 存储器映射与三个外设寄存器的位置

> **【题干】**说明 Cortex-M4 的存储器映射（4GB 地址空间分区），并指出 GPIO、TIM2、CAN 等外设寄存器所在的区域与地址。

Cortex-M4 为 32 位处理器，可寻址 4GB 线性空间（0x0000 0000 ～ 0xFFFF FFFF），ARM 规定按每 512MB 划成 8 个功能区，自低地址起依次为：Code 代码区（片内 Flash、向量表）、SRAM 区、片上外设区 Peripheral、External RAM 区、External Device 区（后两区由 FSMC 外扩使用）、0xE0000000 起的系统级私有区（PPB/调试部件）等。整体划分如图 1 所示：

图 2-1 ARM Cortex-M4 存储器映射（4GB 地址空间划分，★为本题外设）



图 1　ARM Cortex-M4 存储器映射（4GB 地址空间划分，★为本题外设）

题目所问三个外设（以 STM32F407 为例）的寄存器组基址与所属区域如下：

外设寄存器组	基地址	所属区域	所挂总线
GPIOA～GPIOI	0x4002 0000 ～ 0x4002 3FFF	第 4 区 片上外设区 Peripheral	AHB1
TIM2	0x4000 0000	同上	APB1
CAN1 / CAN2	0x4000 6400 / 0x4000 6800	同上	APB1
（对照）SysTick	0xE000 E010	系统级私有区（PPB / SCS）	内核私有总线

> 记忆要点：普通片上外设从 0x4000 0000 开始；内核自己的东西（SysTick、NVIC、SCB）在 0xE000 E000 之后。

第 3 题　中断概念与向量表的作用

> **【题干】**简述中断的处理过程（画简图并标出关键词），并说明异常/中断向量表的作用。

3.1　中断过程简图与关键词

中断响应的完整过程是：主程序正常运行时发生中断请求，NVIC 判优并通过后，硬件自动压栈保护现场、查向量表取得入口地址并执行中断服务程序，最后中断返回、自动出栈恢复现场，流程如图 2：

图 3-1 ARM Cortex-M4 中断响应流程与关键概念

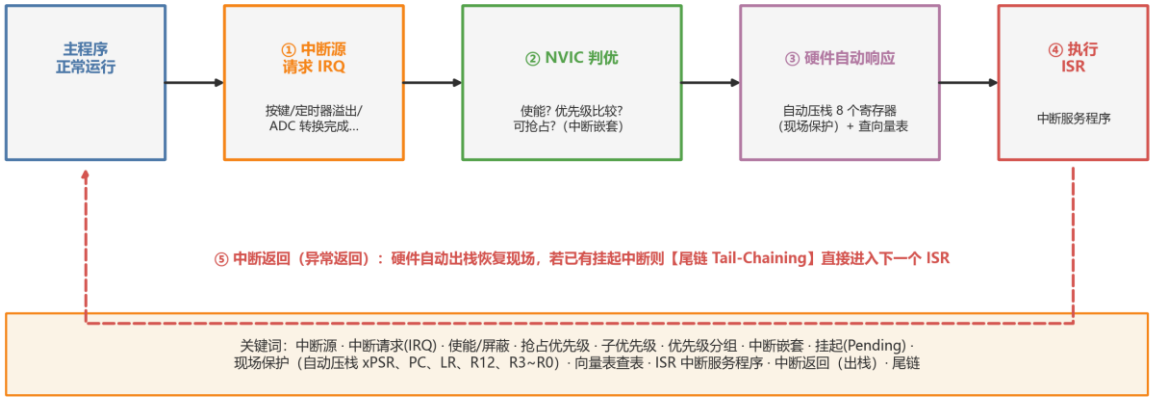


图 2　ARM Cortex-M4 中断响应流程与关键概念

过程中涉及的关键词：中断源、中断请求（IRQ）、中断使能/屏蔽、优先级（抢占优先级、响应/子优先级）、中断优先级分组、中断嵌套、中断挂起（Pending）、现场保护（自动压栈 xPSR、PC、LR、R12、R3～R0）、向量表查表、中断服务程序（ISR）、中断返回（出栈）、尾链（Tail-Chaining）、事件（Event）。

3.2　异常/中断向量表的作用

向量表本质是一张中断服务程序入口地址表，每一表项 4 字节（一个字），按异常号顺序排列，存放在 Code 区（Cortex-M 默认表基址 0x0000 0000，可用 VTOR 寄存器重定位），其结构与作用如图 3：

图 3-2 异常/中断向量表的结构与作用（以 STM32F407 为例，共 98 项）

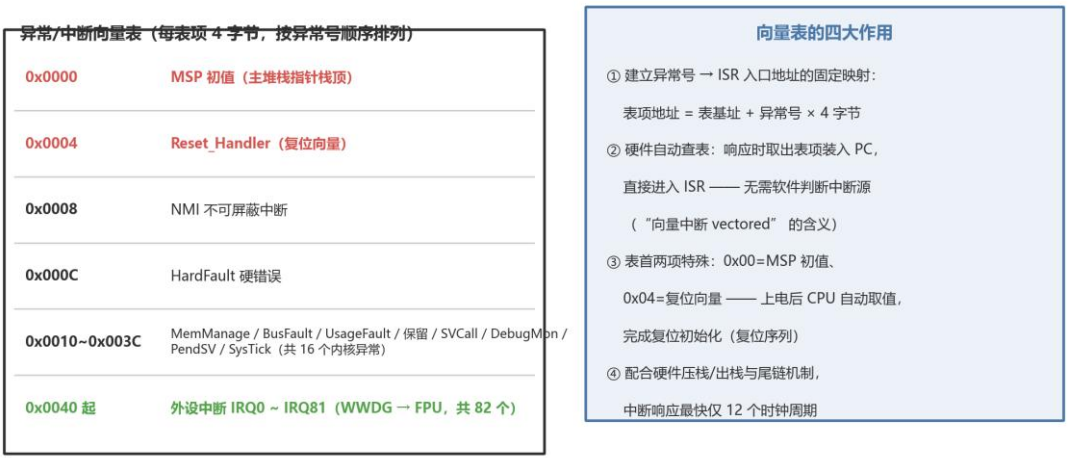


图 3　异常/中断向量表的结构与作用（以 STM32F407 为例，共 98 项）

具体作用有以下四点：

1. 建立中断号到服务程序入口地址的对应关系；

2. 硬件自动查表跳转：异常/中断被响应时，NVIC 把异常号乘以 4 得到表项地址，取出表项内容自动装入 PC，从而进入相应的 ISR——无需软件判断中断源，这就是“向量（vectored）”的含义；

3. 表的第 0 项不是地址而是主堆栈指针 MSP 的初始值，第 1 项是复位向量 Reset_Handler——即复位后 CPU 自动从 0x00000000 取 MSP、从 0x00000004 取 PC，实现复位初始化；

4. 有了向量表与 NVIC，配合硬件压栈/出栈和尾链机制，中断响应最快仅 12 个时钟周期。

向量表结构与启动文件 startup_stm32f40_41xxx.s 中 __Vectors 一致：

表项偏移	内容
0x0000	MSP 初始值（栈顶 __initial_sp）
0x0004	复位 Reset_Handler
0x0008 ～ 0x003C	NMI、HardFault、MemManage、BusFault、UsageFault、保留、SVCall、DebugMon、PendSV、SysTick
0x0040 起	外设中断 IRQ0～IRQ80（WWDG→FPU，共 81 个，F407 总计 98 项）

第 4 题　STM32F40x 片上外设主从总线及三个模块的挂载

> **【题干】**说明 STM32F40x 片上外设的主总线与从总线各有哪几条，并指出 GPIO、TIM2、USART1 分别挂在哪条总线上。

STM32F40x 内部采用多级 AHB/APB 总线矩阵结构（8 条主总线 × 7 条从总线），允许内核与 DMA 并行访问不同的从设备，如图 4：

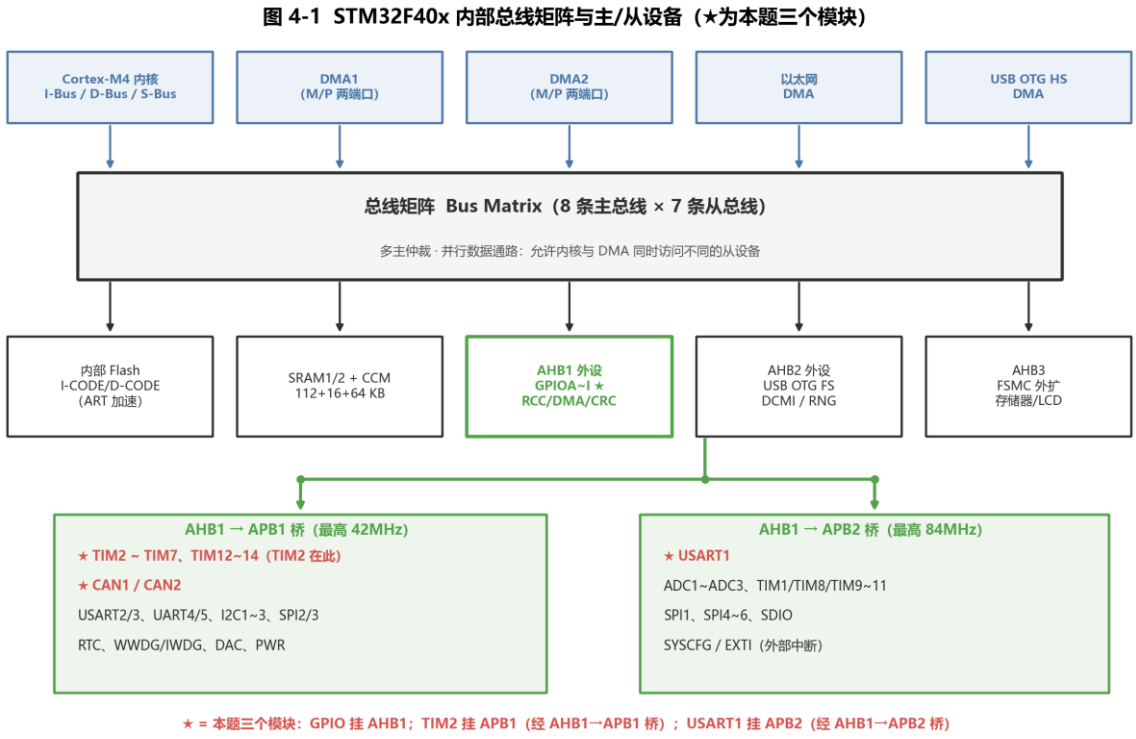


图 4　STM32F40x 内部总线矩阵与主/从设备（★为本题三个模块）

主总线（Master，发起读写的设备）共 8 条：

1. Cortex-M4 内核的三条总线：I-Bus（取指）、D-Bus（取数/系统）、S-Bus（系统访问外设/SRAM）；

2. DMA1、DMA2（各有存储器口和外设口两个主端口，共 4 条）；

3. USB OTG HS 的内部 DMA 主端口；

4. Ethernet（以太网）DMA 主端口。

从总线（Slave，被读写的目标）共 7 条：

1. 内部 Flash（含指令/数据接口，经 ART 加速器）；

2. 内部 SRAM（112KB SRAM1 + 16KB SRAM2 + 64KB CCM）；

3. AHB1→APB1 桥、AHB1→APB2 桥（进而挂接全部 APB 外设，算 2 条）；

4. AHB1 外设（GPIOA～I、CRC、DMA、RCC、Ethernet、USB OTG HS 等）；

5. AHB2 外设（USB OTG FS、相机接口 DCMI、RNG）；

6. FSMC（外部存储器/LCD 控制器，AHB3）。

题目所问三个模块的挂载位置：

模块	所挂总线	使能时钟宏（标准库）
GPIO（GPIOA～GPIOI）	AHB1	`RCC_AHB1Periph_GPIOx`
TIM2	APB1（经 AHB1→APB1 桥）	`RCC_APB1Periph_TIM2`
USART1	APB2（经 AHB1→APB2 桥）	`RCC_APB2Periph_USART1`

> 记忆技巧：GPIO 要高速翻转，挂 AHB1；TIM2 是通用定时器走低速 APB1；USART1 是高速串口走 APB2。

第 5 题　STM32F40x 时钟源与系统时钟

> **【题干】**STM32F40x 有哪些时钟源？SYSCLK 如何产生？AHB1、APB1、APB2 三条总线的最高频率各是多少？

5.1　四个时钟源

类别	名称	频率	用途
高速	HSI	16MHz 内部 RC	上电默认时钟，可不接外部晶振工作
高速	HSE	4～26MHz 外部晶振（板上 8MHz）	主时钟源，精度高，供 PLL
低速	LSI	约 32kHz 内部 RC	独立看门狗 IWDG、RTC 可选
低速	LSE	32.768kHz 外部晶振	RTC 实时时钟（32768=2^{15}，分频恰为 1Hz）

另有主 PLL、I2S-PLL、SAI-PLL 三个锁相环用于倍频合成（IRC48M 专供 USB/RNG）。

5.2　SYSCLK 的产生

SYSCLK 可在 HSI、HSE、PLLCLK 三个源中用 SW 开关选择其一（最高 168MHz）。典型路径为 HSE 经主 PLL 倍频：HSE 8MHz 经 ÷M(=8) 得 1MHz，×N(=336) 得 VCO 336MHz，再 ÷P(=2) 得 168MHz 作 SYSCLK，随后 AHB 分频 ÷1 得 HCLK 168MHz、APB1 分频 ÷4 得 PCLK1 42MHz、APB2 分频 ÷2 得 PCLK2 84MHz，整棵时钟树如图 5：

图 5-1 STM32F40x 时钟树：5 路时钟源 → PLL → SYSCLK → 总线分级

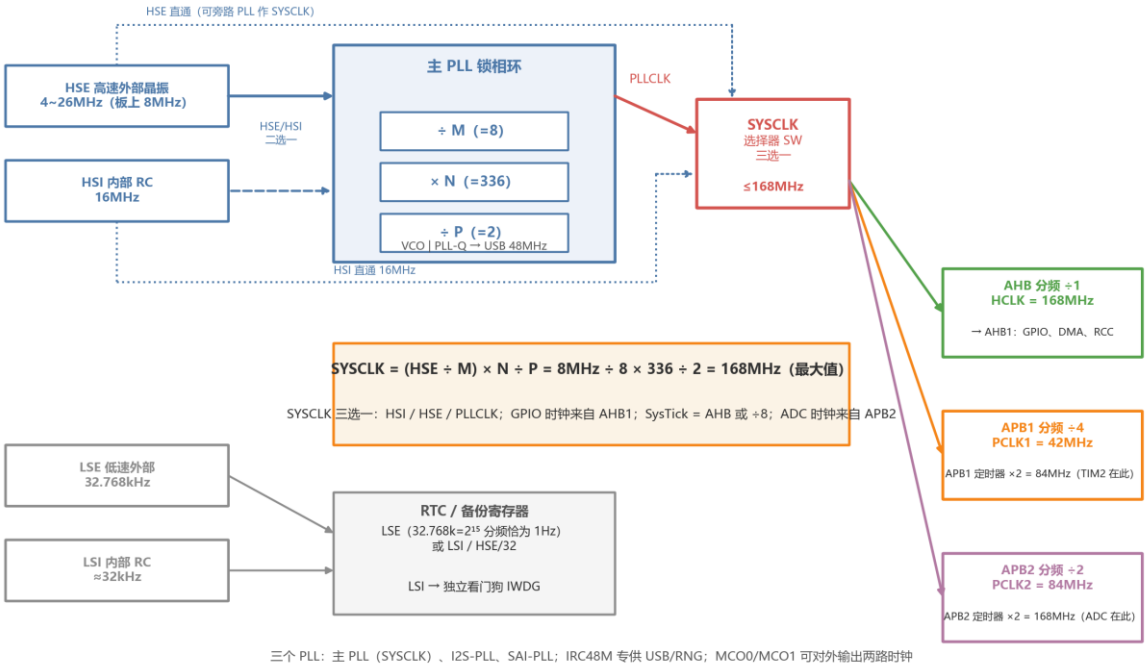


图 5　STM32F40x 时钟树：5 路时钟源 → PLL → SYSCLK → 总线分级

5.3　三条总线最高频率

总线	最高频率	定时器时钟（分频≠1 时 ×2）
AHB1（HCLK）	168 MHz	—
APB1（PCLK1）	42 MHz	84 MHz（TIM2 在此）
APB2（PCLK2）	84 MHz	168 MHz

> 注意 TIM2 挂在 APB1，但 APB1 分频≠1 时定时器时钟自动 ×2（=84MHz）；程序里 SystemInit/SystemClock_Config 就是在配置上面这条 PLL 路径。

第 6 题　复位引脚与典型复位电路

> **【题干】**STM32F407 的复位引脚是哪个？画出典型复位电路并说明其工作原理。

6.1　复位脚：NRST（STM32F407ZGT6 LQFP144 封装第 25 脚）

NRST = Negative Reset，低电平有效的复位引脚（N 表示 Negative，低有效）。它是双向引脚：内部自带约 40kΩ 弱上拉，既可输入（外部电路/按键使芯片复位），也可输出（内部 POR/PDR、看门狗等复位时 NRST 会拉低，供外部芯片同步复位）。只要 NRST 上的低电平持续 ≥20μs，芯片即复位：寄存器回到默认值，随后从 0x00000000 取 MSP 初值、从 0x00000004 取复位向量开始运行。

6.2　典型复位电路（上电复位 + 手动复位，ST 官方推荐参数）

图 6-1 NRST 典型复位电路（上电复位 + 手动复位，ST 官方推荐参数）

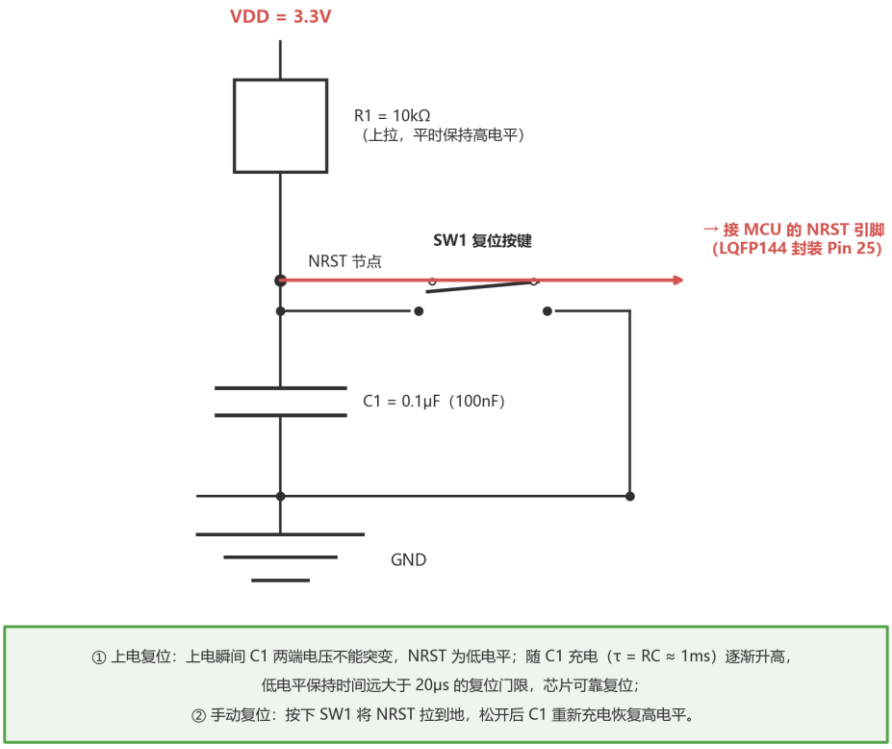


图 6　NRST 典型复位电路（上电复位 + 手动复位，ST 官方推荐参数）

工作原理：

1. C1=100nF 与 R1=10kΩ 构成上电复位：上电瞬间 C1 两端电压不能突变，NRST 为低；随 C1 充电（时间常数 τ≈RC=1ms）NRST 逐渐上升到高电平，保证芯片获得足够长的复位脉冲；

2. 按键 SW1 并联在 C1 上，按下时把 NRST 拉到地，松开后电容充电恢复高电平，实现手动复位；

3. 复位脚走线尽量短、靠近芯片，并避免静电/干扰直接耦合（必要时 NRST 上加 TVS 或靠近芯片放小电容）。

第 7 题　两种开发板主控芯片的特征

> **【题干】**查阅手册，说明两种开发板主控芯片（STM32F407ZGT6 与潘多拉 IoT 板的 STM32L475VET6）的主要特征并作比较。

7.1　STM32F407ZGT6（407 通用板主芯片）

依据官方数据手册 DS8626 Rev12：

1. 内核：ARM Cortex-M4F（32 位，带单精度 FPU、DSP 指令、MPU），最高主频 168MHz，性能 210 DMIPS（1.25 DMIPS/MHz），带 ART 自适应实时加速器（Flash 零等待取指）；

2. 存储：1MB Flash + 192KB SRAM（112+16+64KB CCM 内核耦合存储器）+ 4KB 备份 SRAM；

3. 封装与 IO：LQFP144，114 个 GPIO（Z 表示 144 脚，G 表示 1MB Flash，T=LQFP，6=温度 −40～85℃）；

4. 时钟与电源：1.8～3.6V 供电，HSI/HSE/LSI/LSE + PLL；

5. 外设：3×12 位 ADC（2.4Msps，24 通道）、2×12 位 DAC、17 个定时器（TIM1/TIM8 高级、TIM2～5/9～14 通用、2 看门狗 + SysTick）、4 USART + 2 UART、3 SPI、3 I²C、2×CAN、USB OTG FS + USB OTG HS、以太网 MAC、SDIO、数字摄像头接口 DCMI、真随机数 RNG。

7.2 STM32L475VET6（潘多拉 IoT 开发板主芯片）

依据 ST 官方 STM32L475xx 数据手册与 RT-Thread 官方潘多拉板资料：

1. 内核：ARM Cortex-M4F，最高 80MHz，100 DMIPS——主频比 F407 低，但属于超低功耗（Ultra-low-power）系列，供电 1.71～3.6V；

2. 存储：512KB Flash + 128KB SRAM（E 表示 512KB Flash，V 表示 100 脚，T=LQFP100，6=温度 −40～85℃），82 个 GPIO；

3. 超低功耗特性：多种低功耗模式（低功耗运行、Sleep、Stop1/Stop2、Standby、Shutdown），Stop2 模式约 1μA 级、Standby 约 300nA 级，支持动态调压与低功耗外设（LPUART、LPTIM 等可在低功耗模式下继续工作驱动 RTC）；

4. 外设：3×12 位 ADC（支持硬件过采样提高精度）、2×12 位 DAC、5 个串口（3 USART + 2 UART）、3 SPI、3 I²C、USB OTG FS、SDMMC、Quad-SPI、真随机数 RNG 等；

5. 潘多拉板定位：板载 16MB SPI Flash（W25Q128）、AP6181 WiFi 模块、六轴传感器、AHT10 温湿度传感器、光环境传感器、音频解码、TFTLCD、板载 ST-LINK V2.1 等物联网资源，适合做低功耗联网实验。

一句话对比：F407ZGT6 是“高性能型”（168MHz、丰富通信接口、以太网/双 USB/CAN×2，适合工控与电机）；L475VET6 是“超低功耗型”（80MHz、μA 级待机、电池供电友好，适合 IoT 终端）。两者同为 Cortex-M4F，编程模型一致，学完 F407 可平滑迁移。

> 说明：潘多拉板所用主控以 RT-Thread 官方板卡资料（stm32l475-atk-pandora）核实为 STM32L475VET6；同系列更高配的 STM32L496（1MB Flash/320KB SRAM）特性类似，亦为超低功耗 Cortex-M4F。

第 8 题　扩展练习

8.1　STM32F407ZGT6 最小系统电路图

> **【题干】**① 找到一张 STM32F407ZGT6 最小系统电路图。

采用普中 F407-T100 开发板原理图（主芯片 U1 即 STM32F407ZGT6，A3 整图，完整原图见同目录 PDF），其最小系统的五个组成部分如下：

模块	图中位置	电路要点
电源 POWER	左中部	USB 5V → 1117-3.3 LDO → 3.3V；去耦电容若干
时钟	CPU 下方	8MHz 主晶振（22pF×2 + 1MΩ 并联电阻）+ 32.768kHz RTC 晶振（10pF×2）
复位 RESET	底部	10kΩ 上拉 + 0.1μF + 复位按键（与第 6 题电路一致）
启动配置 BOOT	底部	BOOT0/BOOT1 经 10kΩ 电阻跳线选择启动方式（Flash/系统存储器/SRAM）
下载调试 JTAG	底部	20 针 JTAG 座（含 SWDIO/SWCLK，可 SWD 两线下载）

其余为扩展资源：CH340C USB 转串口、W25Q16 SPI Flash、24C02 EEPROM、TF 卡座、NRF24L01 座、LED、按键、2×27 排针 IO 引出。整图见下页图 7。

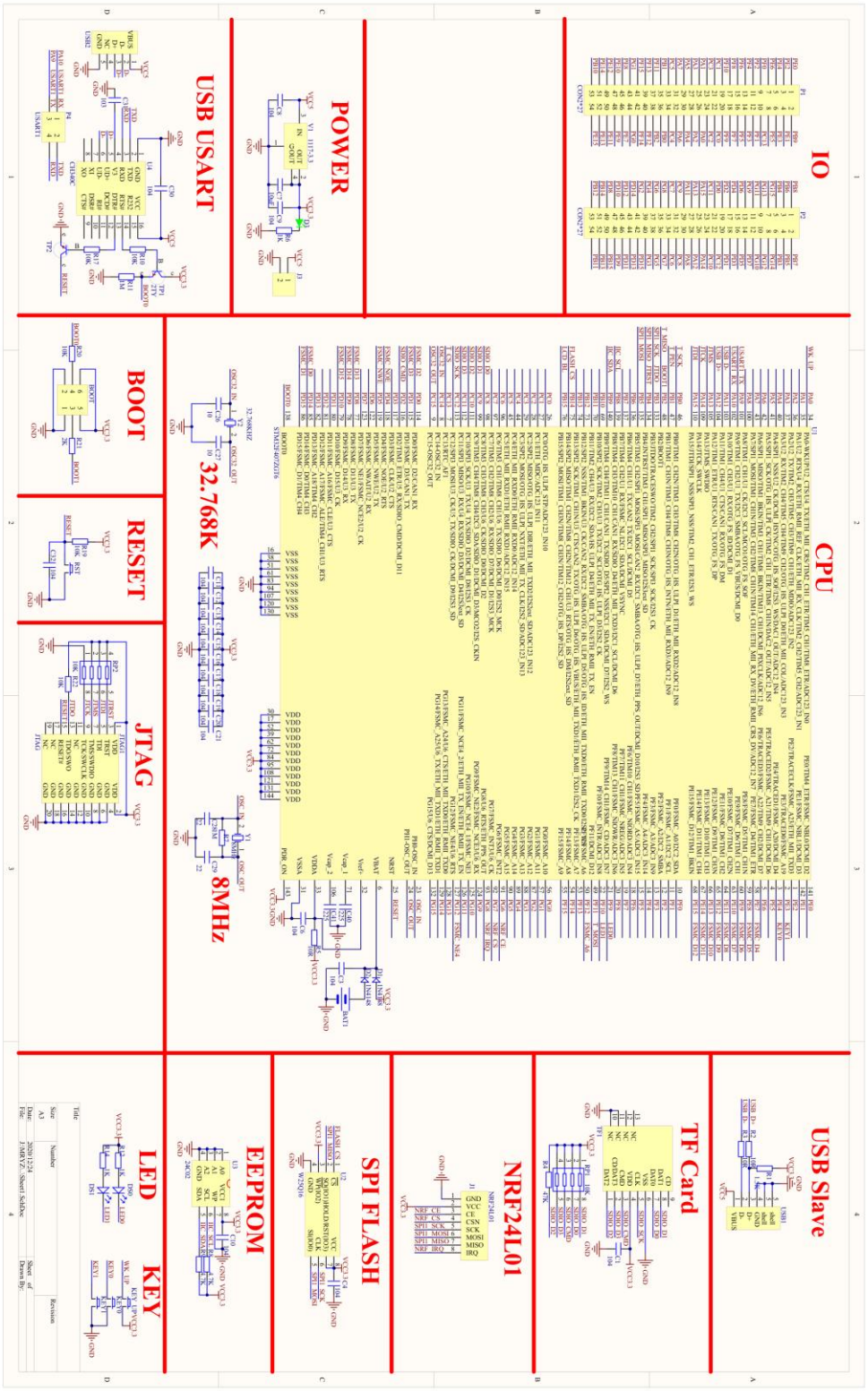


图 7　STM32F407ZGT6 最小系统及外扩资源电路图（普中 F407-T100 开发板，A3 整图竖排）

8.2　标准库 / HAL（抽象）库空白工程要点

建立工程后观察到：

1. Group（分组）典型为：User（main.c、stm32f4xx_it.c、system_stm32f4xx.c）、Startup （ startup_stm32f40_41xxx.s ） 、 CMSIS （ core_cm4.h 、 stm32f4xx.h 、system_stm32f4xx.c、时钟配置）、FWLIB/StdPeriph（标准库：stm32f4xx_gpio.c、stm32f4xx_rcc.c 等，按需添加）或 Drivers/STM32F4xx_HAL_Driver（HAL 库）；

2. main.c 模板结构：包含头文件 → 外设初始化函数（NVIC、GPIO、USART 等 Init ） → while(1) 主循环； HAL 版还有 SystemClock_Config() 、 HAL_Init() 、MX_xxx_Init() 及 Error_Handler()；

3. 模板的作用：把启动文件、时钟配置、中断框架、目录结构和编译环境预先配好，开发者只需在固定位置添加业务代码——避免每次从零搭建工程出错，保证工程可移植、可复现。

8.3 启动文件 startup_stm32f40_41xxx.s 观察记录

1. 堆栈定义： Stack_Size EQU 0x00000400 （ 1KB 栈 ） 、 Heap_Size EQU 0x00000200（512B 堆），SPACE 伪指令保留空间，末地址记为 __initial_sp；

2. 向量表 __Vectors：第 0 项 DCD __initial_sp（MSP 初值），第 1 项 DCD Reset_Handler，其后 16 个内核异常 + 81 个外设中断（IRQ0 WWDG → 最后一项 FPU_IRQHandler），共 98 个 DCD 表项——印证第 3 题向量表结构；

3. 复位序列：上电 → 硬件从 0x00000000 取 MSP 、从 0x00000004 取 PC=Reset_Handler → Reset_Handler 先调用 SystemInit（配置时钟/向量表重定位）→ 再执行“LDR R0, =__main; BX R0”跳入 C 库初始化（搬运初始化数据段/清零 bss、建立堆栈）→ 最终进入用户的 main()；

4. 其余如 [WEAK] 弱定义（中断未实现时跳默认死循环）、EXPORT __initial_sp 供 RTOS 使用等。

8.4 GPIO 工程要点（标准库 vs HAL 库）

操作	标准库（StdPeriph）	HAL 库（抽象库）
开时钟	`RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA, ENABLE);`	`__HAL_RCC_GPIOA_CLK_ENABLE();`

初始化	`GPIO_InitTypeDef g;`
	`g.GPIO_Pin=GPIO_Pin_5;`
	`g.GPIO_Mode=GPIO_Mode_OUT;`
	`g.GPIO_OType=GPIO_OType_PP;`
	`g.GPIO_Speed=GPIO_Speed_100MHz;`
	`g.GPIO_PuPd=GPIO_PuPd_NOPULL;`
	`GPIO_Init(GPIOA,&g);`
		`GPIO_InitTypeDef g={0};`
		`g.Pin=GPIO_PIN_5;`
		`g.Mode=GPIO_MODE_OUTPUT_PP;`
		`g.Pull=GPIO_NOPULL;`
		`g.Speed=GPIO_SPEED_FREQ_HIGH;`
		`HAL_GPIO_Init(GPIOA,&g);`

写引脚	`GPIO_SetBits(GPIOA,GPIO_Pin_5); / GPIO_ResetBits(...);`	`HAL_GPIO_WritePin(GPIOA,GPIO_PIN_5,GPIO_PIN_SET);`

操作	标准库（StdPeriph）	HAL 库（抽象库）
读引脚	`GPIO_ReadInputDataBit(GPIOA,GPIO_Pin_0);`	`HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0);`
翻转	`GPIO_ToggleBits(GPIOA,GPIO_Pin_5);`	`HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_5);`

体会：两者流程相同（先开时钟 → 配结构体 → 初始化 → 读写）；HAL 库还要求先调 HAL_Init() 与 SystemClock_Config()，并引入 HAL_Delay() 等基于 SysTick 的服务，抽象层次更高、可移植性更好，标准库则更直接、资源占用更小。

参考资料

1. 普中 F407-T100 开发板原理图（STM32F407ZGT6，A3 整图）——第 8① 题电路图来源；

2. ST 官方数据手册 DS8626 Rev12《STM32F407ZGT6》——第 6、7 题依据；

3. ST 官方《STM32L475xx Ultra-low-power》数据手册及产品页、RT-Thread 官方潘多拉板（stm32l475-atk-pandora）BSP 说明——第 7 题 L475VET6 依据；

4. 启动文件 startup_stm32f40_41xxx.s（标准库模板）——第 8③ 题观察对象；

5. 课程课件第 3 章《Cortex-M4 内核体系结构》、第 4 章《STM32 芯片应用基础》（教师版）——第 1～6 题知识来源。